

## **-- BDR - Atividade Aula 11**

**-- Aluno: Ronaldo de Avila Ribeiro Junior**

### **-- EXERCÍCIO 1**

-- Listar cada autor, quantos livros publicou e média de páginas

SELECT

A.id\_autor,

A.nome\_autor,

( SELECT COUNT(\*)

FROM public.Livro L

WHERE L.id\_autor = A.id\_autor ) AS quantidade\_livros,

( SELECT AVG(L.paginas)

FROM public.Livro L

WHERE L.id\_autor = A.id\_autor ) AS media\_paginas

FROM public.Autor A;

-- VERSÃO B: CTE

WITH livros\_por\_autor AS (

SELECT

id\_autor,

COUNT(\*) AS quantidade\_livros,

```

        AVG(paginas) AS media_paginas

FROM public.Livro

GROUP BY id_autor

)

SELECT

    A.id_autor,

    A.nome_autor,

    LPA.quantidade_livros,

    LPA.media_paginas

FROM public.Autor A

LEFT JOIN livros_por_autor LPA ON LPA.id_autor = A.id_autor;

```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure, including Schemas (1) and public. The main window displays a SQL query in the Query editor, which is a CTE query. The query is as follows:

```

-- VERSÃO B: CTE
WITH livros_por_autor AS (
    SELECT
        id_autor,
        ( SELECT COUNT(*)
          FROM public.Livro L
          WHERE L.id_autor = A.id_autor ) AS quantidade_livros,
        ( SELECT AVG(L.paginas)
          FROM public.Livro L
          WHERE L.id_autor = A.id_autor ) AS media_paginas
    FROM public.Autor A;
)

```

The Data Output tab shows the results of the query, displaying 5 rows of data. The columns are id\_autor (integer), nome\_autor (character varying (100)), quantidade\_livros (bigint), and media\_paginas (numeric). The results are as follows:

| id_autor | nome_autor         | quantidade_livros | media_paginas                      |
|----------|--------------------|-------------------|------------------------------------|
| 1        | Frank Herbert      | 1                 | 412.000000000000000000000000000000 |
| 2        | J.R.R. Tolkien     | 1                 | 509.000000000000000000000000000000 |
| 3        | Andrzej Sapkowski  | 2                 | 304.000000000000000000000000000000 |
| 4        | Bram Stoker        | 1                 | 418.000000000000000000000000000000 |
| 5        | George R.R. Martin | 2                 | 523.000000000000000000000000000000 |

The status bar at the bottom indicates "Total rows: 5" and "Query complete 00:00:00.130". The system tray shows the date and time as 14/06/2026, 22:23.

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including Schemas (1) and public. The main pane shows a SQL query being executed. The query is as follows:

```

103      GROUP BY id_autor
104    )
105    SELECT
106      A.id_autor,
107      A.nome_autor,
108      LPA.quantidade_livros,
109      LPA.media_paginas
110    FROM public.Autor A
111    LEFT JOIN livros_por_autor LPA ON LPA.id_autor = A.id_autor;
112
113
114
115  -- EXERCÍCIO 2

```

The Data Output pane shows the results of the query, displaying 5 rows. The columns are: id\_autor (integer), nome\_autor (character varying (100)), quantidade\_livros (bigint), and media\_paginas (numeric). The results are as follows:

| id_autor | nome_autor         | quantidade_livros | media_paginas        |
|----------|--------------------|-------------------|----------------------|
| 1        | Frank Herbert      | 1                 | 412.0000000000000000 |
| 2        | J.R.R. Tolkien     | 1                 | 509.0000000000000000 |
| 3        | Andrzej Sapkowski  | 2                 | 304.0000000000000000 |
| 4        | Bram Stoker        | 1                 | 418.0000000000000000 |
| 5        | George R.R. Martin | 2                 | 523.0000000000000000 |

The status bar at the bottom indicates "Total rows: 5", "Query complete 00:00:00.149", and "Ln 111, Col 61".

## -- EXERCÍCIO 2

-- Autores cuja soma de páginas ultrapassa a média geral

WITH paginas\_por\_autor AS (

SELECT

id\_autor,

SUM(paginas) AS total\_paginas

FROM public.Livro

GROUP BY id\_autor

)

SELECT

A.id\_autor,

```

A.nome_autor,

PPA.total_paginas

FROM public.Autor A

JOIN paginas_por_autor PPA ON PPA.id_autor = A.id_autor

WHERE PPA.total_paginas > (

    SELECT AVG(paginas) FROM public.Livro

);

```

The screenshot shows the pgAdmin 4 interface. On the left is the Object Explorer showing the database structure. The main pane displays a SQL query in the 'Query' tab. The query is as follows:

```

-- EXERCÍCIO 2
-- Autores cuja soma de páginas ultrapassa a média geral

WITH paginas_por_autor AS (
    SELECT
        id_autor,
        SUM(paginas) AS total_paginas
    FROM public.Livro
    GROUP BY id_autor
)
SELECT
    A.id_autor,

```

Below the query editor, the 'Data Output' tab shows the results of the query. It displays a table with 3 rows and 3 columns: id\_autor, nome\_autor, and total\_paginas.

| id_autor | nome_autor         | total_paginas |
|----------|--------------------|---------------|
| 1        | J.R.R. Tolkien     | 509           |
| 2        | Andrzej Sapkowski  | 608           |
| 3        | George R.R. Martin | 1046          |

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.133'.

### -- EXERCÍCIO 3

-- Duas versões: subconsulta correlacionada e CTE pré-agrupada

-- VERSÃO A: Subconsulta correlacionada

SELECT

A.id\_autor,

A.nome\_autor,

( SELECT SUM(L.paginas)

FROM public.Livro L

WHERE L.id\_autor = A.id\_autor ) AS total\_paginas

FROM public.Autor A

WHERE

( SELECT SUM(L.paginas)

FROM public.Livro L

WHERE L.id\_autor = A.id\_autor ) > ( SELECT AVG(paginas)

FROM public.Livro );

-- VERSÃO B: CTE pré-agrupada

WITH paginas\_por\_autor AS (

SELECT

id\_autor,

SUM(paginas) AS total\_paginas

FROM public.Livro

GROUP BY id\_autor

)

SELECT

A.id\_autor,

A.nome\_autor,

PPA.total\_paginas

FROM public.Autor A

JOIN paginas\_por\_autor PPA ON PPA.id\_autor = A.id\_autor

WHERE PPA.total\_paginas > (

SELECT AVG(paginas) FROM public.Livro

);

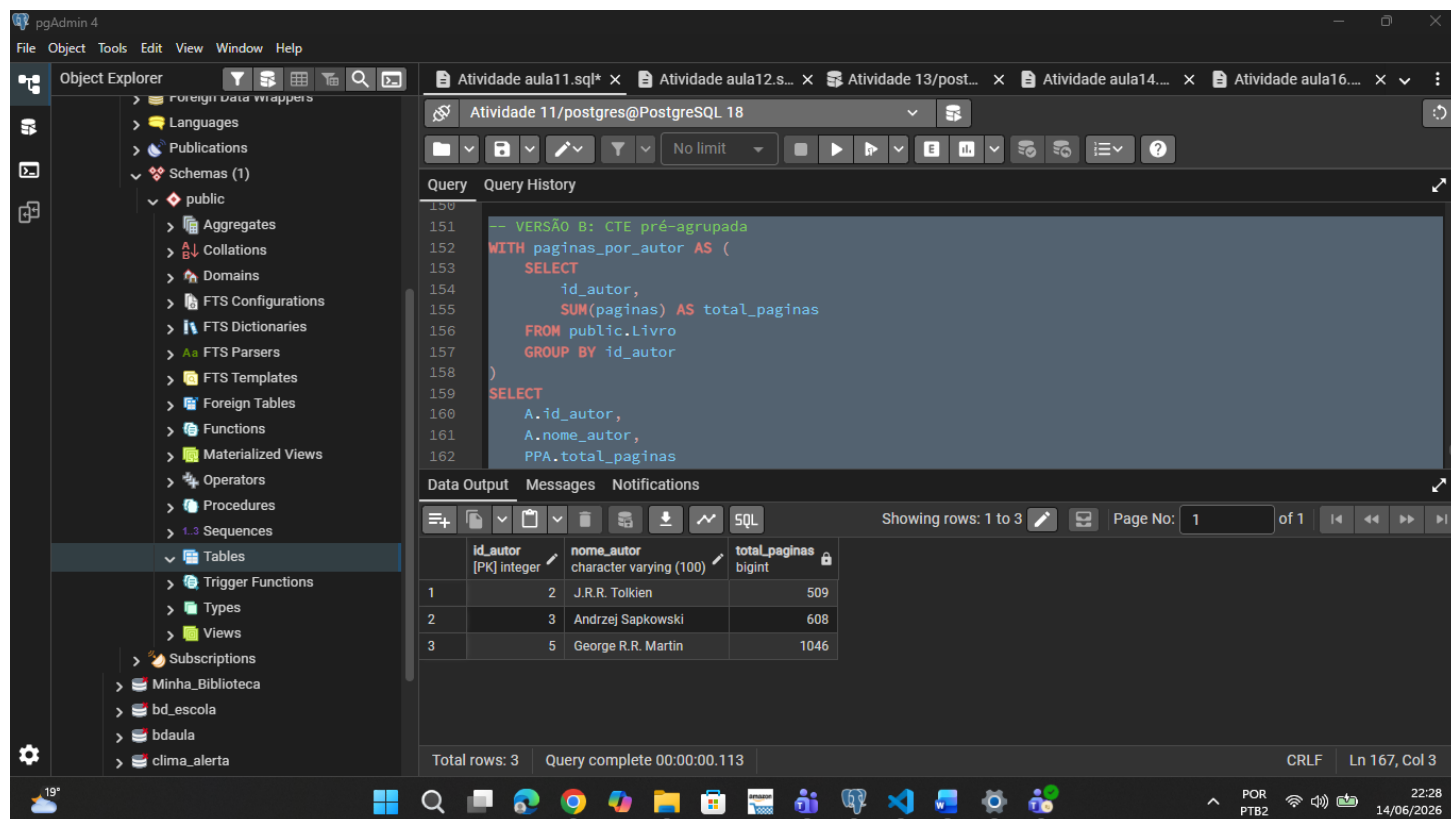
The screenshot shows the pgAdmin 4 interface. On the left is the Object Explorer showing the database structure. The main pane displays a SQL query in the 'Query' tab. The query is a correlated subquery that filters authors based on their total page count compared to the average page count of all books.

```
-- VERSÃO A: Subconsulta correlacionada
SELECT
  A.id_autor,
  A.nome_autor,
  ( SELECT SUM(L.paginas)
    FROM public.Livro L
    WHERE L.id_autor = A.id_autor ) AS total_paginas
FROM public.Autor A
WHERE
  ( SELECT SUM(L.paginas)
    FROM public.Livro L
    WHERE L.id_autor = A.id_autor ) > ( SELECT AVG(paginas) FROM public.Livro );
```

Below the query editor, the 'Data Output' tab shows the results of the query. It displays a table with 3 rows and 3 columns: id\_autor, nome\_autor, and total\_paginas.

| id_autor | nome_autor         | total_paginas |
|----------|--------------------|---------------|
| 1        | J.R.R. Tolkien     | 509           |
| 2        | Andrzej Sapkowski  | 608           |
| 3        | George R.R. Martin | 1046          |

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.115'.



--código completo --

-- BDR - Atividade Aula 11

-- Aluno: Ronaldo de Avila Ribeiro Junior

```
CREATE TABLE IF NOT EXISTS Autor (
  id_autor INT PRIMARY KEY,
  nome_autor VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS Editora (
  id_editora INT PRIMARY KEY,
  nome_editora VARCHAR(50) NOT NULL
```

);

```
CREATE TABLE IF NOT EXISTS Livro (  
    id_livro    INT        PRIMARY KEY,  
    titulo     VARCHAR(100) NOT NULL,  
    ano_publicacao VARCHAR(10) NOT NULL,  
    paginas    INT        NOT NULL,  
    id_autor   INT        NOT NULL,  
    id_editora INT        NOT NULL,  
    FOREIGN KEY (id_autor) REFERENCES Autor (id_autor),  
    FOREIGN KEY (id_editora) REFERENCES Editora (id_editora)  
);
```

```
CREATE TABLE IF NOT EXISTS Cliente (  
    id_cliente INT        PRIMARY KEY,  
    nome_cliente VARCHAR(100) NOT NULL,  
    telefone   VARCHAR(25) NOT NULL,  
    email      VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS Emprestimo_livro (  
    id_emprestimo INT    PRIMARY KEY,  
    data_hora     TIMESTAMP NOT NULL,
```



```
id_livro INT NOT NULL,  
id_cliente INT NOT NULL,  
FOREIGN KEY (id_livro) REFERENCES Livro (id_livro),  
FOREIGN KEY (id_cliente) REFERENCES Cliente (id_cliente)  
);
```

```
INSERT INTO Autor (id_autor, nome_autor) VALUES  
(1, 'Frank Herbert'),  
(2, 'J.R.R. Tolkien'),  
(3, 'Andrzej Sapkowski'),  
(4, 'Bram Stoker'),  
(5, 'George R.R. Martin');
```

```
INSERT INTO Editora (id_editora, nome_editora) VALUES  
(1, 'SARAIVA'),  
(2, 'PANINI'),  
(3, 'Grupo Companhia das Letras'),  
(4, 'DarkSide Books'),  
(5, 'Editora Sextante/Arqueiro'),  
(6, 'ATLAS');
```

```
INSERT INTO Livro (id_livro, titulo, ano_publicacao, paginas,  
id_autor, id_editora) VALUES
```

(1, 'DUNA', '1965', 412, 1, 1),  
(2, 'O Senhor dos Anéis: Retorno do Rei', '1955', 509, 2, 2),  
(3, 'The Witcher', '1993', 288, 3, 3),  
(4, 'Drácula: A Ira do Rei Carmesim', '1897', 418, 4, 4),  
(5, 'Game of Thrones', '1996', 694, 5, 5),  
(6, 'FATE/Strange FAKE', '2011', 352, 5, 4),  
(7, 'Fate Apocraphy', '2017', 320, 3, 2);

INSERT INTO Cliente (id\_cliente, nome\_cliente, telefone, email) VALUES

(1, 'Ronaldo de Avila', '(12) 99785-4587', 'ronaldo00@gmail.com'),  
(2, 'Afonso Nunes', '(11)98754-4987', 'afonso12@gmail.com'),  
(3, 'Danilo Pereira', '(12)98745-1023', 'danilo345@gmail.com'),  
(4, 'Adamael Z.', '(12)94587-1574', 'adamael3456@yahoo.com'),  
(5, 'Artoria Pendagron', '(55) 4787-5871', 'camelot813@outlook.com'),  
(6, 'Sigfried', '(879)7848-4811', 'sifg43a@game.com');

INSERT INTO Emprestimo\_livro (id\_emprestimo, data\_hora, id\_livro, id\_cliente) VALUES

(1, '2021-05-14 14:53:01', 2, 3),

```
(2, '2025-02-25 08:45:01', 5, 1),  
(3, '2026-09-01 15:21:58', 3, 6),  
(4, '2017-08-05 19:24:08', 1, 5),  
(5, '2018-01-31 17:16:27', 4, 5),  
(6, '2025-05-08 12:45:41', 7, 2);
```

-- EXERCÍCIO 1

-- Listar cada autor, quantos livros publicou e média de páginas

SELECT

A.id\_autor,

A.nome\_autor,

( SELECT COUNT(\*)

FROM public.Livro L

WHERE L.id\_autor = A.id\_autor ) AS quantidade\_livros,

( SELECT AVG(L.paginas)

FROM public.Livro L

WHERE L.id\_autor = A.id\_autor ) AS media\_paginas

FROM public.Autor A;

-- VERSÃO B: CTE

```
WITH livros_por_autor AS (  
    SELECT  
        id_autor,  
        COUNT(*) AS quantidade_livros,  
        AVG(paginas) AS media_paginas  
    FROM public.Livro  
    GROUP BY id_autor  
)  
SELECT  
    A.id_autor,  
    A.nome_autor,  
    LPA.quantidade_livros,  
    LPA.media_paginas  
FROM public.Autor A  
LEFT JOIN livros_por_autor LPA ON LPA.id_autor = A.id_autor;
```

-- EXERCÍCIO 2

-- Autores cuja soma de páginas ultrapassa a média geral

```
WITH paginas_por_autor AS (  
    SELECT  
        id_autor,
```

```
        SUM(paginas) AS total_paginas
    FROM public.Livro
    GROUP BY id_autor
)
SELECT
    A.id_autor,
    A.nome_autor,
    PPA.total_paginas
FROM public.Autor A
JOIN paginas_por_autor PPA ON PPA.id_autor = A.id_autor
WHERE PPA.total_paginas > (
    SELECT AVG(paginas) FROM public.Livro
);
```

-- EXERCÍCIO 3

-- Duas versões: subconsulta correlacionada e CTE pré-agrupada

-- VERSÃO A: Subconsulta correlacionada

```
SELECT
    A.id_autor,
    A.nome_autor,
```

```
( SELECT SUM(L.paginas)
  FROM public.Livro L
 WHERE L.id_autor = A.id_autor ) AS total_paginas
FROM public.Autor A
WHERE
  ( SELECT SUM(L.paginas)
    FROM public.Livro L
   WHERE L.id_autor = A.id_autor ) > ( SELECT AVG(paginas)
  FROM public.Livro );
```

```
-- VERSÃO B: CTE pré-agrupada
WITH paginas_por_autor AS (
  SELECT
    id_autor,
    SUM(paginas) AS total_paginas
  FROM public.Livro
  GROUP BY id_autor
)
SELECT
  A.id_autor,
  A.nome_autor,
  PPA.total_paginas
FROM public.Autor A
```

```
JOIN paginas_por_autor PPA ON PPA.id_autor = A.id_autor
WHERE PPA.total_paginas > (
    SELECT AVG(paginas) FROM public.Livro
);
```